
ParetoProof: When Does Pareto Selection Help Maintain Lean Proofs?

Adarsh Agrawal
Senior Member, IEEE

Abstract

When a proof system finds several correct proofs, which one should it keep? A short proof may be fragile, while a robust proof may be slower or depend on more of the library. Pareto selection is meant for such tradeoffs, but it can only help when the available proofs actually disagree on the objectives. We study this question in Lean using 649 natural maintenance units with 1,649 valid candidates. Only 160 units (24.7%) have more than one nondominated candidate, and 75 (11.6%) show high objective conflict. Among 312 units with later-version results, PARETOPROOF improves survival over equal-weight scalarization by 3.5 percentage points (78.5% versus 75.0%, $p = 0.0207$), but its 1.3-point advantage over adaptive scalarization is not statistically detectable ($p = 0.424$). A post-hoc analysis suggests that Pareto selection becomes more useful as objective conflict grows, although this pattern requires prospective confirmation. A separate LLM-assisted generation study over 16 theorem families explains why the average benefit is small: even after 32 generations, each theorem has 9.5 distinct verified proofs but only 3.31 nondominated choices on average. Our practical conclusion is simple. When the likely maintenance failure is known, optimize the matching proxy. When objectives mostly agree, adaptive scalarization captures nearly all observed benefit. Conflict-rich pools are the setting in which Pareto selection should be tested prospectively.

1 Introduction

Generating a valid proof is only the first step. A repository must also live with that proof as libraries, tactics, and dependencies change. If several proofs are correct, the best one today may not be the best one to maintain.

This creates a natural multi-objective problem. We may prefer fewer tokens, faster checking, fewer dependencies, less fragile automation, and no warnings. Pareto selection keeps proofs that are not worse on all of these dimensions. The idea is appealing, but it has an easily overlooked requirement: there must be several valid proofs with genuinely different strengths. If one proof dominates the rest, or every objective ranks the proofs similarly, a Pareto method has little to do.

We ask when this requirement holds and whether it matters for later proof survival. Our main evaluation gives several selection policies exactly the same valid candidate pools, then checks the selected proofs on later versions. This separates the choice of proof from the model that generated it. A second study tests Pareto retention during LLM-assisted generation at matched call budgets.

The answer is conditional. Pareto selection improves on fixed equal weights, while a well-adapted scalar score remains statistically unresolved within a few percentage points. The clearest potential opportunity for Pareto selection appears in the relatively small set of candidate pools where objectives strongly disagree. When the expected failure mode is already known, directly optimizing the matching proxy works just as well.

Contributions.

1. We evaluate proof selection independently of proof generation using 649 maintenance units and 1,649 valid candidates.
2. We show that the opportunity for Pareto selection is uncommon: 24.7% of units have more than one nondominated candidate, and only 11.6% have high conflict.
3. We turn the results into a practical decision heuristic among direct proxy optimization, adaptive scalarization, and prospective Pareto retention.

2 Related Work

Learning-based theorem provers combine language models with retrieval, native checking, search, and repair. LeanDojo exposes repository-scale Lean interaction [3]; Baldur studies whole-proof generation and repair [5]; and generative proof search and informal guidance have been studied across theorem-proving settings [7, 9]. Benchmarks such as miniF2F and ProofNet evaluate formal mathematical reasoning [11, 13], while ITPEval provides a controlled cross-prover evaluation surface [1]. These works mainly evaluate whether a proof is produced. We study which verified proof should be retained for later maintenance.

Pareto methods such as NSGA-II preserve nondominated populations instead of fixing one tradeoff [15]. Search-based software engineering similarly treats quality as multi-dimensional [17]. Our focus is diagnostic: whether the candidate pools and downstream outcomes in proof maintenance satisfy the conditions that make Pareto retention useful.

3 ParetoProof

3.1 Verified candidate pools

For maintenance unit i , let C_i be a fixed candidate set and let $V_i \subseteq C_i$ contain the candidates accepted by Lean. We only allow changes to the proof body. The theorem statement, imports, declarations, and surrounding source remain unchanged.

Each valid candidate receives five within-unit normalized utilities, oriented so that higher is better:

$$\mathbf{u}_i(c) = [u_{\text{tok}}, u_{\text{elab}}, u_{\text{dep}}, u_{\text{frag}}, u_{\text{warn}}].$$

They measure token efficiency, elaboration efficiency, dependency minimality, fragility resistance, and warning cleanliness. We call an objective a maintenance proxy only when it is associated with a mechanically independent future outcome. The same fixed within-unit normalization is used by every selector, and no comparison depends on utilities from different maintenance units sharing an absolute scale.

3.2 Selection policies

Candidate c dominates c' when it is no worse on all five objectives and better on at least one. The nondominated set is

$$\mathcal{F}_i = \{c \in V_i : \nexists c' \in V_i \text{ with } c' \succ c\}.$$

PARETOPROOF selects the candidate in $\mathcal{F}_i$ closest to the normalized ideal point:

$$c_i^{\text{P}} = \arg \min_{c \in \mathcal{F}_i} \|\mathbf{1} - \mathbf{u}_i(c)\|_2.$$

The strongest comparator is an adaptive scalarizer, $\arg \max_c \sum_a w_{ia} u_{ia}(c)$, whose normalized weights respond to objective variation within the candidate pool. The primary fixed baseline uses $w_{ia} = 1/5$ for all objectives. A sensitivity analysis uses weights (0.30, 0.30, 0.20, 0.10, 0.10) for tokens, elaboration, dependencies, fragility, and warnings. We also evaluate a fixed lexicographic ordering and uniform random-valid selection. All policies operate on the same frozen candidates.

3.3 Geometry and conflict

Pareto selection has a distinct choice only when $|V_i| \geq 2$ and $|\mathcal{F}_i| \geq 2$. We also measure continuous conflict

$$d_i = \frac{1}{\binom{5}{2}} \sum_{a < b} \frac{1 - \rho_{iab}}{2},$$

Table 1: Frozen-pool composition. The primary endpoint has one paired all-version outcome per future-covered unit.

Source	Units	Future units	Attempts
Mathlib	318	201	566
Lean-Gym snapshots	156	67	134
miniF2F	113	32	64
ProofNet	62	12	12
Total	649	312	776

where ρ_{iab} is the rank correlation between objectives a and b over the candidates in unit i . Larger d_i means stronger ranking disagreement. Descriptively, *any conflict* requires at least two objective disagreements above the 20% rule in the frozen analysis; *high conflict* requires at least three above the 30% rule. Only the continuous score enters the moderator model.

4 Evaluation

4.1 Natural maintenance study

The main study contains 649 maintenance units and 1,649 valid candidates from four Lean source collections. One unit is a theorem with its maintained proof and a fixed set of alternatives. Later-version results are available for 312 units, covering 776 unit-version checks. The candidate pools are frozen before the future outcomes are evaluated, and this study uses no LLM inference. Table 1 summarizes the data.

We treat these as source collections rather than four independent repositories. Exact snapshot identifiers and future-version boundaries are not included in this public preprint and remain limitations of the current release.

Primary endpoint and tests. A selected proof succeeds only if it survives every available later version for its unit, yielding one paired binary outcome per unit. Exact McNemar tests compare PARETOPROOF with adaptive and fixed-equal scalarization. These two comparisons form the primary family, so we use $\alpha = 0.025$. Failure to reject is not treated as evidence of equivalence.

Coverage sensitivity. Future coverage differs across the four sources. We therefore report a repository-standardized analysis alongside the observed results. This checks whether the main ordering is driven by sources with better future-version coverage. Full weighting and bootstrap details appear in Appendix B.

Moderator and proxy analyses. After seeing the average selector results, we test whether objective conflict explains where Pareto selection helps. A repeated-measures logistic model includes selector, continuous conflict, their interaction, source, and version index. This analysis is exploratory. Separately, we compare the objective scores with future survival and use an 89-unit upstream-change set to compare PARETOPROOF with direct fragility optimization.

4.2 Controlled generation study

The controlled study compares one-shot generation, iterative repair, scalarized search, and PARETO-PROOF on 16 ITPEval Lean families. Three model configurations, three seeds, four methods, and a three-call budget produce 576 episodes and 1,728 model calls. Lean 4.18 checks each candidate, and Lean 4.33 provides a later-version transfer test. We average model and seed results within each theorem family before statistical testing. Additional budgets of 8, 16, and 32 calls test whether the small frontiers are simply caused by too little generation.

The primary three-call campaign uses Claude Opus 5, Claude Sonnet 5, and Qwen3-Coder 480B-A35B. The powered 8- and 16-call campaigns use Opus 5, Sonnet 5, and GPT-5.6 Terra. The 32-call saturation curve is a single-model Sonnet 5 sweep; a separate public-pool check includes Sonnet 5,

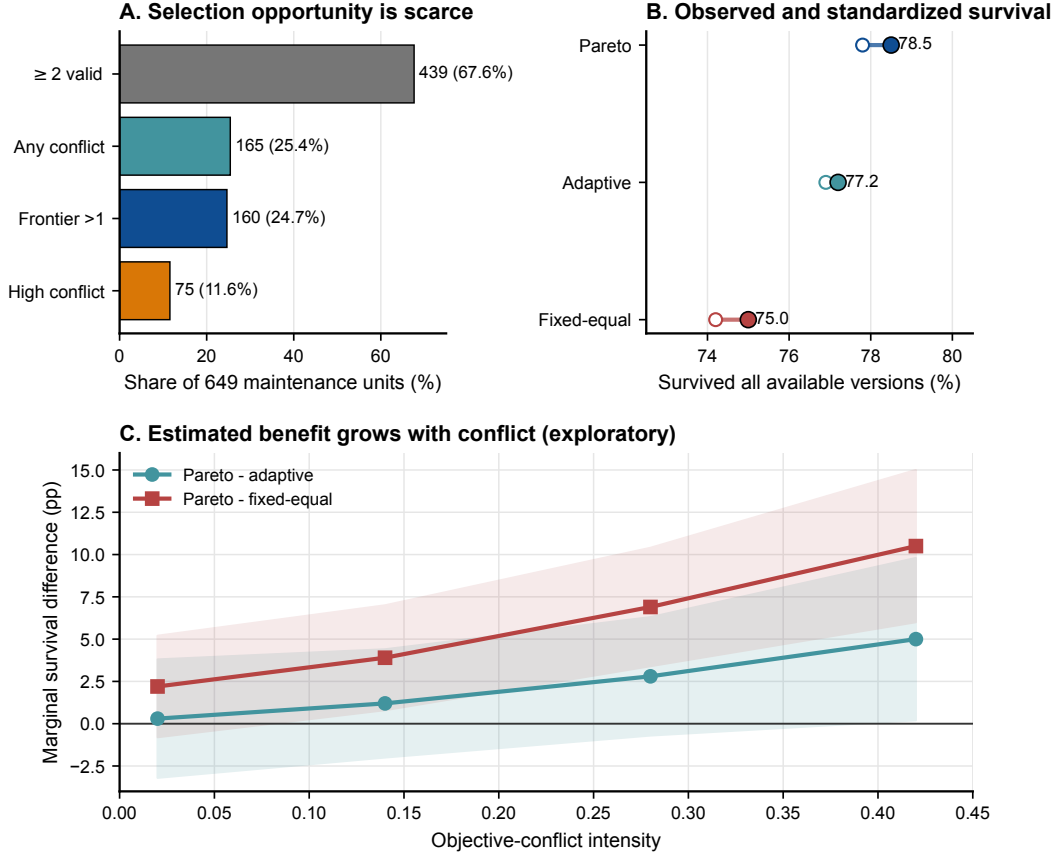


Figure 1: **Geometry and future-version outcomes.** **A:** The funnel from multiple valid candidates to conflict-rich frontiers. **B:** Observed all-version survival for 312 units is shown with filled points; open points are repository-standardized estimates. **C:** Follow-up estimates across continuous conflict. Bands are pointwise 95% intervals; this pattern needs confirmation on new data.

DeepSeek 3.2, Qwen3-Coder, and GPT-5.6 Terra. A three-task directional check uses GPT-5.6 sol. All models are accessed through hosted inference services without training or fine-tuning.

5 Results

5.1 The opportunity for Pareto selection is uncommon

The mean pool contains 2.54 valid candidates, but 210 of 649 units contain only one. Although 439 units have at least two valid candidates, only 160 (24.7%) have a frontier larger than one, 165 (25.4%) exhibit any conflict, and 75 (11.6%) meet the high-conflict criterion. Of the 160 nontrivial-frontier units, 147 also exhibit conflict ($\phi = 0.82$). Candidate multiplicity is therefore not equivalent to meaningful multi-objective choice.

5.2 Pareto improves on fixed weights; adaptive remains unresolved

PARETOPROOF survives all available versions on 245 of 312 units (78.5%). Fixed-equal scalarization survives on 234 (75.0%). The paired difference is 3.5 pp with $p = 0.0207$, meeting the adjusted threshold for the two primary comparisons. Adaptive scalarization survives on 241 units (77.2%), leaving a 1.3 pp difference from PARETOPROOF ($p = 0.424$).

Repository-standardized rates are 77.8% for PARETOPROOF, 76.9% for adaptive, and 74.2% for fixed-equal. The weighted PARETOPROOF-minus-fixed difference is 3.6 pp (95% bootstrap interval [0.4, 6.8] pp, $p = 0.027$); the adaptive difference is 0.9 pp ([-2.3, 4.1] pp, $p = 0.582$). An alternative

Table 2: Primary paired all-version endpoint. Differences are comparator minus PARETOPROOF; tests are exact McNemar comparisons.

Selector	Survival	Difference	Paired p
PARETOPROOF	245/312 (78.5%)	–	–
Adaptive scalar	241/312 (77.2%)	–1.3 pp	0.4240
Fixed-equal scalar	234/312 (75.0%)	–3.5 pp	0.0207

fixed weighting gives 74.4% unweighted survival and a 4.1 pp Pareto advantage ($p = 0.0072$). The standardized fixed-equal estimate preserves the direction and magnitude of the primary result, although its p -value narrowly exceeds the adjusted $\alpha = 0.025$ threshold. The alternative-weight sensitivity supports the same ordering. The adaptive comparison remains unresolved within a few percentage points.

5.3 Conflict may explain when Pareto selection helps

In the follow-up model, the interaction is negative for adaptive scalarization relative to Pareto ($\beta = -0.584$, robust SE 0.285, 95% CI $[-1.142, -0.026]$, $p = 0.041$) and larger for fixed-equal scalarization ($\beta = -1.089$, SE 0.331, 95% CI $[-1.738, -0.440]$, $p = 0.001$). The estimated Pareto advantage over adaptive scalarization is 0.3 pp at the 10th conflict percentile, 1.2 pp at the median, and 5.0 pp at the 90th percentile. Only the last pointwise interval excludes zero, $[0.2, 9.8]$ pp.

The thresholded subgroup shows the same pattern but exposes the uncertainty. Among 46 high-conflict units with future data, survival is 84.8% for Pareto, 78.3% for adaptive, and 71.7% for fixed-equal. The paired p -values are 0.375 and 0.070 because only five and eight pairs are discordant. We therefore do not claim a validated conflict threshold. Instead, the result suggests a clear question for a future evaluation: does a conflict-based rule chosen in advance reproduce this advantage?

5.4 Known threats favor direct optimization

Fragility resistance has the strongest association with future survival ($\rho = 0.67$, 95% CI $[0.59, 0.74]$). Token efficiency and dependency minimality are moderate ($\rho = 0.41$ and 0.38), elaboration is weak (0.22), and warning cleanliness does not validate as a proxy (0.14, CI $[-0.01, 0.28]$). On 89 isolated upstream changes, direct fragility selection survives on 73 units (82.0%) and PARETOPROOF on 72 (80.9%); three pairs favor fragility, two favor Pareto, and $p = 1.0$. Random-valid selection reaches 64.0%. When the downstream threat matches one validated proxy, balancing unrelated objectives adds no measured value.

5.5 Controlled generation reveals frontier scarcity

At budget 3, PARETOPROOF minus scalarized search is +1.5 pp on compression ($p = 0.268$), +0.7 pp on transfer ($p = 1.0$), and –12.6 pp on elaboration reduction ($p = 0.00126$). At budgets 8 and 16, the elaboration disadvantage remains same-signed across three model families and equals –5.2 pp ($p = 0.0037$) and –6.4 pp ($p = 2.0 \times 10^{-5}$). Removing population diversity changes no deployable outcome (all $p \geq 0.78$).

The over-generation study distinguishes verified-candidate scarcity from frontier scarcity. At 32 generations, each theorem has 9.5 distinct verified model proofs on average, but the nondominated set, including the original proof, averages only 3.31. More sampling therefore creates correct alternatives without producing enough objective conflict for population retention to matter.

6 Practical Guidance and Limitations

Decision rule. If the downstream threat is known and a validated proxy measures it, optimize that proxy. If the threat is uncertain but objective rankings mostly agree, use adaptive scalarization. If the threat is uncertain and the pool is conflict-rich, Pareto retention is a candidate for prospective evaluation, not yet an established default.

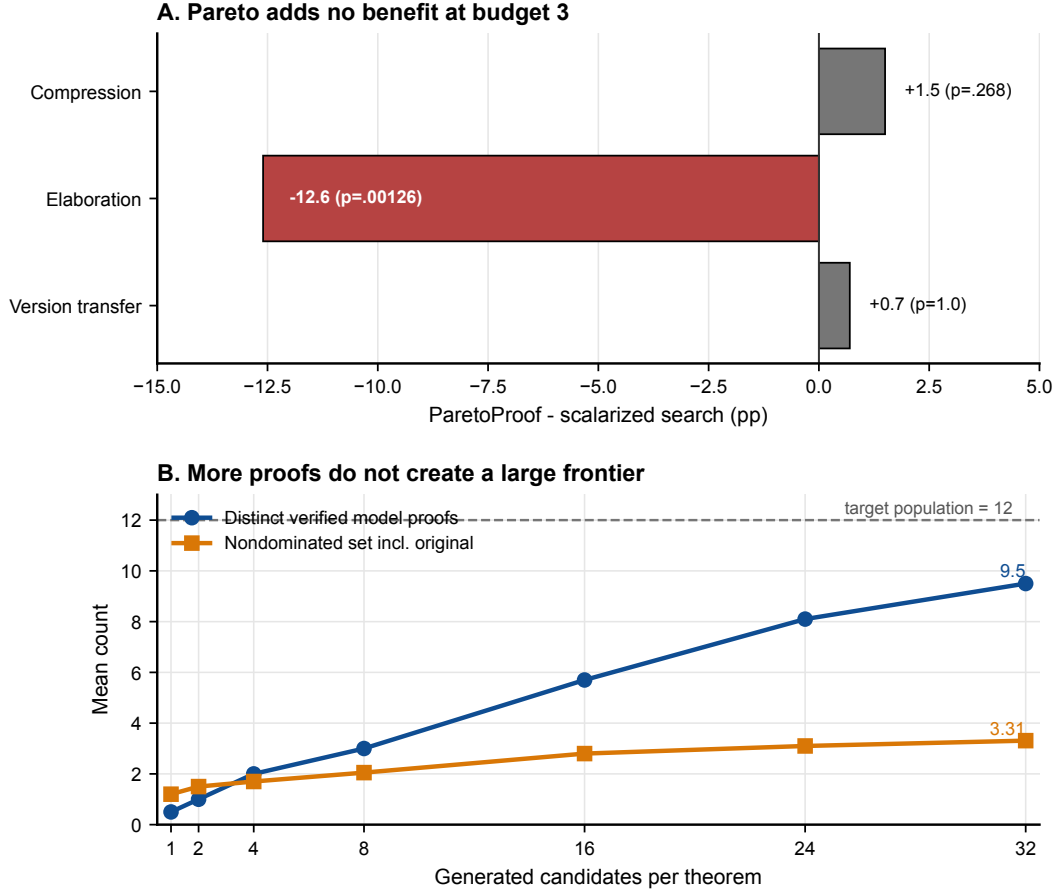


Figure 2: **Controlled generation and saturation.** **A:** At budget 3, PARETOPROOF does not improve compression or transfer over scalarized search and worsens elaboration reduction. **B:** More generations produce verified proofs faster than they produce nondominated alternatives.

Limitations. The natural study covers four Lean source collections, so transport to other proof assistants is unknown. Only 312 of 649 units have future evidence, and source-level weighting cannot correct within-source missingness. Twenty-five Mathlib units become untestable by the last boundary; counting all as failures lowers absolute survival but preserves the ordering. The conflict analysis was motivated after observing the primary result. Future survival does not capture readability, architectural fit, or review cost, and we do not claim that the measured proxies capture every aspect of maintainability. The study also focuses on Lean, so the result may not transfer directly to other proof assistants.

Broader impact. Selecting proofs that survive repository evolution could reduce maintenance work and make formalized software and mathematics more dependable. The main risk is misplaced confidence: a selector may optimize an incomplete proxy and discard a proof that would better fit future needs. Every selected proof should still be checked by Lean, and the choice should be revisited when the expected maintenance threat changes.

7 Conclusion

Pareto selection is useful only after a candidate pool supplies a genuine multi-objective decision. Across 649 Lean maintenance units, that condition is uncommon. Pareto selection improves all-version survival over fixed equal weights, while its difference from a competent adaptive scalarizer is not statistically detectable. A post-hoc analysis suggests that conflict-rich pools may be the exception, while an isolated upstream-change study shows that direct proxy optimization is sufficient when the

threat is known. The controlled generation campaign reaches the same mechanism from another direction: more verified proofs do not necessarily yield a large nondominated frontier. Multi-objective proof systems should report candidate-pool geometry before claiming a selector advantage.

References

- [1] J. Wu, R. J. George, and A. Anandkumar. ITPEval: Benchmarking formal translation across interactive theorem provers. *arXiv preprint arXiv:2607.19407*, 2026.
- [2] S. Indukuri and A. Agrawal. Grounded optimization: A layered engineering framework for reducing LLM hallucination in automated personal document rewriting. *arXiv preprint arXiv:2607.01457*, 2026.
- [3] K. Yang, A. M. Swope, A. Gu, et al. LeanDojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 2023.
- [4] Amazon Artificial General Intelligence. Amazon Nova multimodal embeddings: Technical report and model card. *Amazon Technical Reports*, 2025.
- [5] E. First, M. N. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-proof generation and repair with large language models. *ESEC/FSE*, 2023.
- [6] A. Agrawal and R. S. Babu. Incident memory: Training-free operational memory through sequential pattern mining and velocity-stratified retrieval. *arXiv preprint arXiv:2609.01616*, 2026.
- [7] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- [8] Amazon Artificial General Intelligence. Amazon Nova Sonic: Technical report and model card. *Amazon Technical Reports*, 2025.
- [9] A. Q. Jiang, W. Li, J. M. Han, and Y. Wu. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *International Conference on Learning Representations*, 2023.
- [10] A. Agrawal and S. Indukuri. Schema-first retrieval: Embedding catalogs for natural language analytics. *arXiv preprint arXiv:2606.28387*, 2026.
- [11] K. Zheng, J. M. Han, and S. Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.
- [12] Amazon Artificial General Intelligence. Amazon Nova Premier: Technical report and model card. *Amazon Technical Reports*, 2025.
- [13] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. Ayers, D. Radev, and J. Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.
- [14] R. S. Babu and A. Agrawal. Self-healing agentic orchestrators for reliable tool-augmented large language model systems. *arXiv preprint arXiv:2606.01416*, 2026.
- [15] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [16] Amazon Artificial General Intelligence. Amazon Nova 2: Multimodal reasoning and generation models. *Amazon Technical Reports*, 2025.
- [17] J. Petke, S. O. Haraldsson, M. Harman, W. B. Langdon, D. R. White, and J. R. Woodward. Genetic improvement of software: A comprehensive survey. *IEEE Transactions on Software Engineering*, 44(4):415–432, 2018.
- [18] W. Feng, A. Agrawal, H. Ling, E. Blasch, E. Adiles-Cruz, P. T. Schrader, and J. Wei. DDDAS probability learning for natural disaster change detection. *International Conference on Dynamic Data Driven Applications Systems*, pages 90–99, 2024.

- [19] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. Introducing v0.5 of the AI safety benchmark from MLCommons. *arXiv preprint arXiv:2404.12241*, 2024.
- [20] Amazon Artificial General Intelligence. The Amazon Nova family of models: Technical report and model card. *Amazon Technical Reports*, 2024.
- [21] A. Agrawal and J. Li. Mitigating bias in AI using Debias-GAN. *NVIDIA GTC Conference and WWT Research*, 2022.
- [22] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. Others. 2024. Introducing v0.5 of the AI safety benchmark from MLCommons. *arXiv preprint arXiv:2404.12241*, 2024.
- [23] A. V. Adarsh Agrawal and N. Garg. Time series forecasting of producer price index, using ARIMA. *International Journal of Science and Research*, 7(7):1019–1024, 2018.
- [24] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. How to construct AI safety benchmarks. Manuscript, n.d.
- [25] B. Vidgen, A. Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. Introducing v0.5 of the AI safety benchmark from MLCommons, 2024. Available at <https://arxiv.org/abs/2404.12241>, 2024.

A Paired Outcome Tables

Table 3: Exact paired counts for the primary all-version endpoint.

Outcome over 312 units	Adaptive	Fixed-equal
Both survive	236	229
PARETOPROOF only	9	16
Comparator only	5	5
Neither survives	62	62

Table 4: Exploratory high-conflict counts. Exact paired p -values are 0.375 against adaptive and 0.070 against fixed-equal scalarization.

Outcome over 46 units	Adaptive	Fixed-equal
Both survive	35	32
PARETOPROOF only	4	7
Comparator only	1	1
Neither survives	6	6

B Geometry, Weighting, and Dropout Details

Table 5: Conflict and frontier cross-tabulation. The association is $\phi = 0.82$, but neither definition is constructed from the other.

	Conflict	No conflict	Total
Frontier > 1	147	13	160
Frontier $= 1$	18	471	489
Total	165	484	649

The stabilized source weight is $P(\text{observed})/P(\text{observed} \mid \text{source})$, with numerator 312/649. The resulting weights are 0.761 for Mathlib, 1.121 for Lean-Gym, 1.700 for miniF2F, and 2.479 for ProofNet. Their sum is approximately 312 and their Kish effective sample size is 266.

For the last Mathlib boundary, 25 of the original 201 units become untestable: five theorems are removed, seven are substantially refactored, seven lose the required dependency context, and six encounter validation-infrastructure failure. Treating all 25 as failures gives survival of 73.1% for PARETOPROOF, 70.6% for adaptive, and 65.2% for fixed-equal, preserving the pairwise ordering.

C Mechanical Proxy Validation

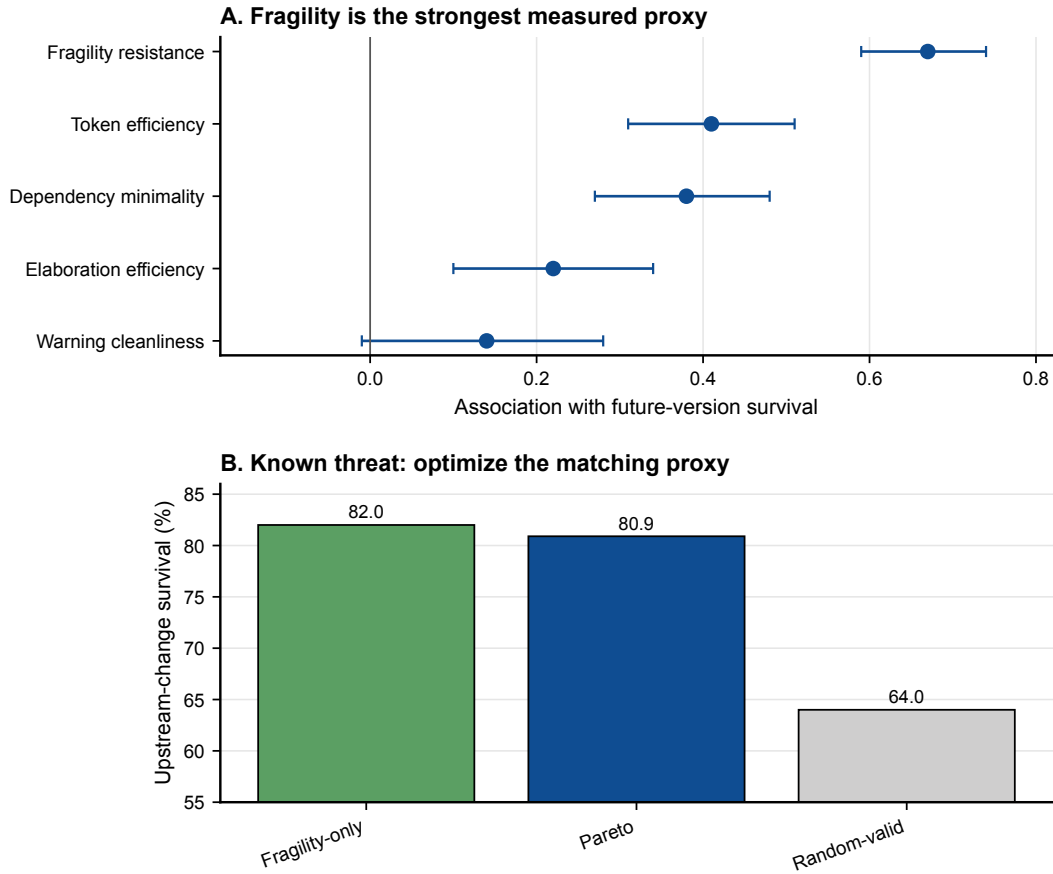


Figure 3: **Mechanical validation of the objective proxies.** **A:** Associations with future-version survival. **B:** On isolated upstream changes, direct fragility optimization matches PARETOPROOF and both exceed random-valid selection.

D Additional Experimental Details

The primary endpoint uses one binary outcome per unit and exact paired tests. The GEE is a separate follow-up model with 2,328 rows, 776 unit-version combinations, and 312 clusters. Its marginal-effect intervals are pointwise rather than simultaneous. Repository-standardized intervals resample units within source strata. A non-significant comparison is never described as equivalence.

D.1 Models and generation

The controlled three-call campaign uses four methods, 16 theorem families, three seeds, and three models: Claude Opus 5, Claude Sonnet 5, and Qwen3-Coder 480B-A35B. The powered 8- and 16-call campaigns replace Qwen3-Coder with GPT-5.6 Terra. The single-model saturation sweep uses Sonnet 5 through 32 calls. A separate public-pool check uses Sonnet 5, DeepSeek 3.2, Qwen3-Coder, and Terra, and a small directional check uses GPT-5.6 sol. Inference was performed through hosted Runtime and Responses-style APIs in August 2026. Claude models use provider reasoning controls without a temperature parameter; GPT-5.6 models use reasoning-effort controls; open-weight model decoding is fixed per campaign. LLMs propose proof bodies, but Lean alone determines validity. Prompt templates and complete request configurations are not included in this public preprint.

D.2 Reproducibility materials

The figures in this preprint were generated from aggregate evidence packets. Candidate-level objective data, selector choices, per-version outcomes, and the complete analysis pipeline are not included in this public release. Regenerating the controlled candidate pools would additionally require access to the listed hosted models.

D.3 Compute resources

Lean verification and statistical analysis run on CPU-only Ubuntu 24.04 x86_64 hosts; no local accelerator is used. LLM inference uses provider-managed hardware. The primary controlled campaign makes 1,728 model calls. Exact RAM, worker count, wall time by campaign, and preliminary-run overhead are not included in this public preprint.

D.4 Existing assets and licenses

Table 6: Versions and licenses of the existing assets used in the experiments.

Asset	Version or snapshot	License
Mathlib	Frozen study snapshot	Apache-2.0
Lean-Gym	Frozen repository snapshot	Apache-2.0
miniF2F	Frozen benchmark snapshot	MIT
ProofNet	Frozen benchmark snapshot	MIT
ITPEval	Frozen Lean-family snapshot	Mirror-specific

The cited source papers provide attribution. Exact snapshot identifiers, the ITPEval mirror and corresponding license, and model-service terms are not included in this public preprint and remain limitations of the current release.